

Bellman Policy Optimization

Zhuoqing Song^{1,2}, Haotian Xu¹, Xikun Zhang¹, and Lidong Bing¹

¹Apodex US, Inc. ²Princeton University

Abstract

Reinforcement learning with verifiable rewards (RLVR) improves the reasoning capabilities of large language models (LLMs). We introduce Bellman Policy Optimization (BPO), a critic-free method derived from Policy Mirror Descent (PMD). For autoregressive generation with terminal rewards, BPO uses the Bellman equations to reformulate PMD as a trajectory-level objective. The reformulation avoids estimating state values at intermediate states. We prove that it has the same unique optimal solution as the original PMD objective. We derive the practical BPO loss by approximating this objective. Its mismatch-correction weight is a smoothed ratio of complementary token probabilities. Experiments on mathematical reasoning benchmarks demonstrate the effectiveness of BPO.

1 Introduction

Reinforcement learning with verifiable rewards (RLVR) has become an important approach for improving the reasoning capabilities of large language models (LLMs) [1, 2, 3, 4, 5, 6]. In RLVR, models are trained using outcome-level rewards assigned to generated responses by task-specific verifiers. Recent work has shown that choices in policy optimization can have a substantial impact on both reasoning performance and training stability [4, 7, 8, 9, 10, 11].

Group-Relative Policy Optimization (GRPO) and its variants are widely used in RLVR [1, 4, 7, 8, 12, 13, 14, 15]. GRPO samples multiple responses for each prompt and computes advantages by normalizing their rewards within the group. Under outcome supervision, all tokens in a response share the same advantage. GRPO uses these advantages in a PPO-style clipped surrogate objective with token-level importance-sampling ratios [16]. This formulation and its variants avoid training a value model and support large-scale reasoning-model training [2, 3, 5, 13, 17, 18, 19].

We start from Policy Mirror Descent (PMD) [20, 21]. PMD updates the policy using action values and a divergence penalty. However, directly applying this update to language generation requires value estimates at intermediate states. A common approach is to train a separate value model, which increases memory and computational costs [1]. Learned value estimates can also be inaccurate on reasoning tasks [22]. We seek a reformulation of PMD that recovers the same policy update without training a value model.

We introduce Bellman Policy Optimization (BPO), a critic-free policy optimization method derived from PMD. Our derivation builds on prior work that reparameterizes rewards and values using policy likelihood ratios [11, 23, 24, 25]. For autoregressive generation with terminal rewards, the Bellman equations express each token advantage as a difference between value functions at consecutive states. These differences telescope along a response, leaving only the terminal reward and the initial value. We combine this identity with the PMD optimality condition to derive a

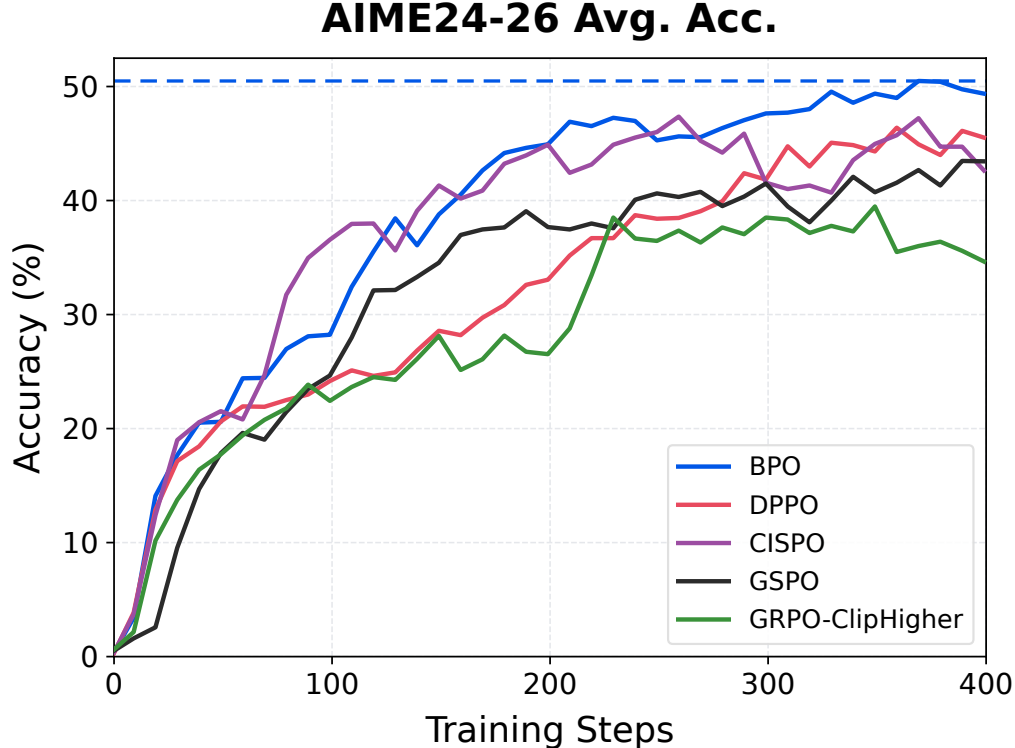


Figure 1: Evaluation accuracy during training on Qwen3-30B-A3B-Base, comparing BPO with GRPO-ClipHigher, GSPO, CISPO, and DPPO. All methods are trained for 400 training steps on the English subset of DAPO-Math-17k under identical experimental settings except for the policy loss. Each rollout batch contains 256 prompts, with a group size of 16 responses per prompt and a maximum response length of 16384 tokens. One training step consists of one rollout batch followed by eight optimizer updates. Curves show mean Avg@32 accuracy across AIME24–26, with 32 responses sampled per question to estimate Pass@1. The horizontal dashed line marks BPO’s peak mean accuracy.

trajectory-level objective. The initial value is the expected reward for a prompt and can be estimated from sampled responses. The reformulation therefore avoids estimating values or advantages at intermediate states. We prove that this objective and the original PMD objective have the same unique optimal solution on states reachable under the rollout policy.

We then derive the practical BPO loss through a sequence of approximations. We estimate the initial value using the mean reward of responses sampled for each prompt. We approximate the gradient of the trajectory-level objective and replace the full KL divergence with a binary approximation [8]. The resulting gradient includes a mismatch-correction weight determined by the rollout and current token probabilities. We apply additive smoothing to this weight for numerical stability. In the practical BPO loss, this weight replaces the importance-sampling ratio used in GRPO.

We evaluate BPO on Qwen3-30B-A3B-Base trained on DAPO-Math-17k. We compare against GRPO-ClipHigher [1, 4], GSPO [7], CISPO [13], and DPPO [8] under identical experimental settings. As shown in Figure 1, BPO achieves a peak average accuracy of 50.5% across AIME 2024–2026 [26]. The gains over these four baselines are 11.0, 7.0, 3.1, and 4.1 percentage points, respectively.

Our main contributions are:

- We derive a critic-free, trajectory-level reformulation of PMD for autoregressive generation with terminal rewards, avoiding value or advantage estimation at intermediate states. We prove that this reformulation and the original PMD objective have the same unique optimal solution on states reachable under the rollout policy.
- We derive the practical BPO loss from this reformulation through group-based estimation and gradient approximations. The loss replaces GRPO’s importance-sampling ratio with a mismatch-correction weight given by a smoothed ratio of complementary token probabilities.
- We evaluate BPO on mathematical reasoning benchmarks using Qwen3-30B-A3B-Base. BPO achieves a peak average accuracy of 50.5% across AIME 2024–2026, outperforming GRPO-ClipHigher, GSPO, CISPO, and DPPO by 3.1–11.0 percentage points.

2 Preliminaries

2.1 Notation

Let $\mathcal{V}$ denote the vocabulary. Let x denote a prompt and let $y = (y_1, \dots, y_T)$ denote a response. Both x and y are token sequences over $\mathcal{V}$. In response y , y_t is the t -th token and the length T may vary across responses. We also use $|y|$ to denote its length.

For $1 \leq t \leq u \leq T$, let $y_{t:u} = (y_t, \dots, y_u)$ denote the subsequence of y from positions t through u . We write $y_{<t} = y_{1:t-1}$, $y_{\leq t} = y_{1:t}$, and $y_{\geq t} = y_{t:T}$.

For any token sequences $z_1, \dots, z_k$, $(z_1, \dots, z_k)$ denotes their concatenation. For instance, $(x, y_{<t})$ denotes the prompt x followed by the response subsequence $y_{<t}$. For a function whose argument is a token sequence, we write $f(z_1, \dots, z_k)$ as shorthand for $f((z_1, \dots, z_k))$ whenever no ambiguity arises.

2.2 RLVR as an Episodic MDP

Reinforcement learning with verifiable rewards (RLVR) for large language models (LLMs) can be formulated as a finite-horizon episodic Markov Decision Process (MDP).

Let D denote a prompt set and let $\mathcal{D}$ denote a prompt distribution on D . We assume $\mathcal{D}$ is uniform on D for simplicity. The analysis extends directly to more general prompt distributions.

States and transitions. For a prompt $x \in D$, the initial state s_1 is defined as the sequence x itself. At step $t > 1$, the state s_t concatenates the prompt x and the generated prefix $y_{<t}$: $s_t = (x, y_{<t})$. The transition is deterministic: appending token y_t to $s_t = (x, y_{<t})$ yields the next state $s_{t+1} = (x, y_{\leq t})$.

Policies and completion distributions. Let $\Delta(\mathcal{V}) = \{p : \mathcal{V} \rightarrow [0, 1] \mid \sum_{v \in \mathcal{V}} p(v) = 1\}$ denote the set of probability distributions on $\mathcal{V}$. A stochastic policy is a mapping $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{V})$. In particular, $\pi(\cdot \mid x, y_{<t}) \in \Delta(\mathcal{V})$ denotes the next-token distribution at position t . The policy space is denoted by $\Pi = \Delta(\mathcal{V})^{\mathcal{S}}$. For any policy π , let $\Pi_\pi \subseteq \Pi$ denote the set of policies that are absolutely continuous with respect to π .

For any policy $\pi \in \Pi$ and state $s_t \in \mathcal{S}$, we use $\mathbb{P}_\pi(\cdot \mid s_t)$ to denote the completion distribution induced by π , i.e., $\mathbb{P}_\pi(y_{\geq t} \mid s_t) = \prod_{u=t}^{|y|} \pi(y_u \mid s_u)$, where $y_{\geq t} = y_{t:|y|}$ as mentioned in Section 2.1.

The action at step t is the t -th token y_t in the generated response, and is sampled from the policy $\pi(\cdot \mid x, y_{<t})$. In practice, π is parameterized by an autoregressive LLM. The response $y = (y_1, \dots, y_T)$ terminates when the EOS token is generated or the response length limit $T_{\max}$ is reached.

Rewards and objective. A deterministic verifier R assigns the terminal reward. Without loss of generality, we assume $|R(x, y)| \leq 1$ for any prompt-response pair (x, y) .

The policy objective is

$$\mathcal{J}(\pi) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \mathbb{P}_\pi(\cdot|x)} [R(x, y)]. \quad (1)$$

2.3 Group-Relative Policy Optimization

In practical RLVR, responses are generated by a rollout policy μ , while policy optimization is performed on the current policy π . Policy updates between rollout generation and training steps, together with numerical discrepancies between the rollout and training engines, can cause the two policies to differ [8, 27, 28, 29, 30]. Group-Relative Policy Optimization (GRPO) [1] is a critic-free, PPO-style policy optimization method [16]. For each prompt $x \sim \mathcal{D}$, GRPO independently samples a group of G responses $\{y^i\}_{i=1}^G$ from $\mathbb{P}_\mu(\cdot|x)$. Let $R_i = R(x, y^i)$ denote the reward of response y^i . GRPO assigns each response the group-normalized advantage

$$\hat{A}^i = \frac{R_i - \text{mean}(\{R_j\}_{j=1}^G)}{\text{std}(\{R_j\}_{j=1}^G)}. \quad (2)$$

GRPO applies PPO-style clipping to the token-level importance ratio. For the t -th token in response y^i , its per-token loss is defined as

$$\mathcal{L}_{i,t}^{\text{GRPO}}(\pi) = -\min \left\{ r_t^i \hat{A}^i, \text{clip}(r_t^i, 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) \hat{A}^i \right\}, \quad (3)$$

where r_t^i is the token-level importance ratio defined as

$$r_t^i = \frac{\pi(y_t^i | s_t^i)}{\mu(y_t^i | s_t^i)}. \quad (4)$$

The following per-token loss has the same gradient as (3) and is therefore equivalent for gradient-based training:

$$\tilde{\mathcal{L}}_{i,t}^{\text{GRPO}}(\pi) = -\hat{A}^i M_t^i \text{sg}(r_t^i) \log \pi(y_t^i | s_t^i), \quad (5)$$

where the clipping mask M_t^i is defined as

$$M_t^i = \begin{cases} 0, & \hat{A}^i > 0 \text{ and } r_t^i > 1 + \epsilon_{\text{high}}, \\ 0, & \hat{A}^i < 0 \text{ and } r_t^i < 1 - \epsilon_{\text{low}}, \\ 1, & \text{otherwise.} \end{cases} \quad (6)$$

2.4 Bellman Equations with Terminal Rewards

We next introduce value functions and Bellman equations [31] for the terminal-reward setting described in Section 2. For a fixed policy π and state $s_t = (x, y_{<t})$, define the value function as the expected reward under policy π given state s_t , i.e.,

$$V^\pi(s_t) = \mathbb{E}_{z \sim \mathbb{P}_\pi(\cdot|s_t)} [R(x, (y_{<t}, z))], \quad (7)$$

where the expectation is taken over all completions under policy π given state $s_t = (x, y_{<t})$. The initial value $V^\pi(s_1) = V^\pi(x)$ is therefore the expected reward for prompt x under policy π .

Since intermediate rewards are zero, for a non-terminal state s_t , the action-value function is defined as

$$Q^\pi(s_t, y_t) = \mathbb{E}_{s' \sim \mathbb{P}(\cdot | s_t, y_t)} [V^\pi(s')] = V^\pi(s_{t+1}), \quad (8)$$

where $s_{t+1} = (s_t, y_t)$ and the second equality follows from the deterministic transition described in Section 2.

For a non-terminal state s_t , the advantage function is defined as

$$A^\pi(s_t, y_t) = Q^\pi(s_t, y_t) - V^\pi(s_t). \quad (9)$$

The advantage function $A^\pi(s_t, y_t)$ measures how favorable the current action y_t is at state s_t relative to the expected action value under policy $\pi(\cdot | s_t)$. It follows from (7) and (9) that

$$\sum_{y_t \in \mathcal{V}} \pi(y_t | s_t) A^\pi(s_t, y_t) = 0. \quad (10)$$

For any non-terminal state s_t , the Bellman equation is

$$V^\pi(s_t) = \sum_{y'_t \in \mathcal{V}} \pi(y'_t | s_t) Q^\pi(s_t, y'_t) = \sum_{s' = (s_t, y'_t): y'_t \in \mathcal{V}} \pi(y'_t | s_t) V^\pi(s').$$

For a complete response y , the terminal state is $s_{|y|+1} = (x, y)$. The terminal condition is

$$V^\pi(s_{|y|+1}) = R(x, y).$$

2.5 Policy Mirror Descent

Let $\mathcal{S}$ denote the state space, and let $\mathcal{V}$ denote the vocabulary as in Section 2. Given a real-valued function $f : \mathcal{S} \times \mathcal{V} \rightarrow \mathbb{R}$, Policy Mirror Descent (PMD) [20, 21, 32, 33] solves the following optimization problem for each state $s \in \mathcal{S}$:

$$\max_{\pi(\cdot | s) \in \Delta(\mathcal{V})} \mathbb{E}_{y \sim \pi(\cdot | s)} [f(s, y)] - \frac{1}{\eta} D_{\text{KL}}(\pi(\cdot | s) \| \mu(\cdot | s)), \quad (11)$$

where μ is the behavior policy (rollout policy) and $\eta > 0$ is the step size.

The optimization problem (11) has the unique solution

$$\pi_{\mu, f}^+(y | s) = \frac{\mu(y | s) \exp(\eta f(s, y))}{Z_{\mu, f}(s)}, \quad (12)$$

where the partition function is

$$Z_{\mu, f}(s) = \mathbb{E}_{y' \sim \mu(\cdot | s)} \exp(\eta f(s, y')).$$

2.6 Binary KL Divergence

Let $\pi, \mu \in \Pi$ be two policies on $\mathcal{V}$. For a fixed state s , their KL divergence is

$$D_{\text{KL}}(\pi(\cdot | s) \| \mu(\cdot | s)) = \sum_{y \in \mathcal{V}} \pi(y | s) \log \frac{\pi(y | s)}{\mu(y | s)}.$$

Computing the full KL divergence requires logits for the entire vocabulary and can be costly.

The binary KL divergence introduced in [8] approximates the full KL divergence. For a given action $y \in \mathcal{V}$, it partitions the action space into $\{y\}$ and its complement $\mathcal{V} \setminus \{y\}$. This induces the following Bernoulli distributions:

$$\pi_y^{\text{bin}}(\cdot|s) = (\pi(y|s), 1 - \pi(y|s)), \quad \mu_y^{\text{bin}}(\cdot|s) = (\mu(y|s), 1 - \mu(y|s)).$$

We define the binary KL divergence associated with action y as

$$\begin{aligned} D_{\text{KL}}^{\text{bin}}(\pi(\cdot|s) \parallel \mu(\cdot|s); y) &= D_{\text{KL}}\left(\pi_y^{\text{bin}}(\cdot|s) \parallel \mu_y^{\text{bin}}(\cdot|s)\right) \\ &= \pi(y|s) \log \frac{\pi(y|s)}{\mu(y|s)} + (1 - \pi(y|s)) \log \frac{1 - \pi(y|s)}{1 - \mu(y|s)}. \end{aligned} \quad (13)$$

As shown in [8], binary KL divergence provides a lower bound for KL divergence, i.e.,

$$0 \leq D_{\text{KL}}^{\text{bin}}(\pi(\cdot|s) \parallel \mu(\cdot|s); y) \leq D_{\text{KL}}(\pi(\cdot|s) \parallel \mu(\cdot|s)), \quad \forall y \in \mathcal{V}.$$

3 Bellman Policy Optimization

We first define the BPO loss and then derive it from PMD.

BPO Loss Definition. For each prompt $x \sim \mathcal{D}$, we independently sample a group of G responses $\{y^i\}_{i=1}^G$. For the t -th token of response y^i , the BPO loss is

$$\mathcal{L}^{\text{BPO}}(\pi) = -\hat{A}^i M_t^i \min\{\text{sg}(\omega_t^i), C\} \log \pi(y_t^i | x, y_{<t}^i), \quad (14)$$

where $\hat{A}^i$ is the advantage of response y^i in the group $\{y^i\}_{i=1}^G$ defined in (2) and the terms ω_t^i , C , and M_t^i are defined below:

- ω_t^i is the mismatch-correction weight for the t -th token in response y^i , defined as

$$\omega_t^i = \frac{1 + \epsilon - \mu(y_t^i | x, y_{<t}^i)}{1 + \epsilon - \pi(y_t^i | x, y_{<t}^i)}, \quad (15)$$

- The constant C caps ω_t^i to stabilize training. The mask M_t^i follows the GRPO clipping rule in (6), with the weight ω_t^i replacing the importance-sampling ratio:

$$M_t^i = \begin{cases} 0, & \hat{A}^i > 0 \text{ and } \omega_t^i > 1 + \epsilon_{\text{high}} \\ 0, & \hat{A}^i < 0 \text{ and } \omega_t^i < 1 - \epsilon_{\text{low}} \\ 1, & \text{otherwise.} \end{cases}$$

The BPO per-token loss retains the GRPO form in (5) and replaces the importance-sampling ratio r_t^i with the truncated mismatch-correction weight $\min\{\text{sg}(\omega_t^i), C\}$.

Derivation of the BPO Objective. We derive the BPO objective (14) in three steps:

- Step 1 (Starting Point, (16)): we instantiate PMD with $f(s, y) = A^\mu(s, y)$, where $A^\mu(s, y)$ is the advantage function introduced in Section 2.4.
- Step 2 (Critic-free PMD, Section 3.1): we derive a critic-free objective with the same optimal solution as advantage-based PMD. This avoids estimating $A^\mu(s, y)$ at individual states.
- Step 3 (Practical Approximation, Section 3.2): we approximate the critic-free PMD objective from Step 2 to obtain the BPO objective (14).

Starting Point: Advantage-Based PMD. We first instantiate the general PMD objective in (11) with the rollout-policy advantage $A^\mu(s, y)$ of the terminal-reward MDP in Section 2.4, i.e., $f(s, y) = A^\mu(s, y)$. Classical PMD is commonly formulated using the action-value function $Q^\mu(s_t, y_t)$ [20, 21, 32, 33]. Using A^μ gives the same update because $Q^\mu(s_t, y_t) = A^\mu(s_t, y_t) + V^\mu(s_t)$, and the term $V^\mu(s_t)$ does not depend on the action y_t .

Advantage-based PMD solves the following problem for each state $s_t = (x, y_{<t})$:

$$\max_{\pi(\cdot|s_t) \in \Delta(\mathcal{V})} \mathbb{E}_{y_t \sim \pi(\cdot|s_t)} [A^\mu(s_t, y_t)] - \frac{1}{\eta} D_{\text{KL}}(\pi(\cdot|s_t) \parallel \mu(\cdot|s_t)). \quad (16)$$

As discussed in Section 2.5, the optimization problem (16) has the unique solution π^+ , given at each state $s_t = (x, y_{<t})$ by

$$\pi^+(y_t|s_t) = \frac{\mu(y_t|s_t) \exp(\eta A^\mu(s_t, y_t))}{Z_\mu(s_t)}, \quad (17)$$

where the partition function is

$$Z_\mu(s_t) = \mathbb{E}_{y'_t \sim \mu(\cdot|s_t)} \exp(\eta A^\mu(s_t, y'_t)).$$

Here, we use π^+ and Z_μ as abbreviations for π_{μ, A^μ}^+ and Z_{μ, A^μ} .

3.1 Critic-Free Reformulation of Policy Mirror Descent

Directly implementing the update (16) requires estimating $A^\mu(s_t, y_t)$ at every visited intermediate state. This typically requires a learned critic or costly conditional rollouts. We derive the critic-free reformulation in (18) by expressing rewards and values through policy likelihood ratios and the Bellman equations. This follows prior work on direct alignment [11, 23, 24, 25].

Critic-Free Reformulation of PMD. Let μ denote the rollout policy. We assign each prompt $x \in D$ a fixed positive weight $\phi(x)$. We will specify this weight in Section 3.2 to obtain the practical BPO loss. We consider the following critic-free objective:

$$\min_{\pi \in \Pi_\mu} \mathcal{L}(\pi) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \mathbb{P}_\mu(\cdot|x)} \left[\phi(x) \cdot \frac{\delta(x, y; \pi, \mu)^2}{2\eta} \right], \quad (18)$$

where the trajectory-level residual $\delta(x, y; \pi, \mu)$ is defined as

$$\delta(x, y; \pi, \mu) = \eta(R(x, y) - V^\mu(x)) - \sum_{t=1}^{|y|} \left(\log \frac{\pi(y_t|s_t)}{\mu(y_t|s_t)} + D_{\text{KL}}(\mu(\cdot|s_t) \parallel \pi(\cdot|s_t)) \right). \quad (19)$$

The reformulation (18) does not depend on state-value functions at intermediate states. Instead, it only requires the terminal reward $R(x, y)$ and the initial value function $V^\mu(x)$. The terminal reward $R(x, y)$ is directly provided by the verifier, while $V^\mu(x) = \mathbb{E}_{y \sim \mathbb{P}_\mu(\cdot|x)} [R(x, y)]$ is the expected reward for prompt x under the rollout policy μ . We can estimate this expectation from rollout data.

Equivalence to PMD. Theorem 1 shows that the critic-free objective (18) and PMD (16) have the same unique optimal solution. This equivalence holds for any positive weighting function ϕ . We specify ϕ in Section 3.2 when deriving the practical BPO loss (14).

Theorem 1. *Let the policy π^+ in (17) denote the optimal solution of the PMD problem (16). Let $\phi : D \rightarrow \mathbb{R}_+$ be any positive weight function on the prompt set D . For any prompt $x \in D$, we solve the optimization problem (18). Then, this optimization problem admits a unique optimal solution $\hat{\pi}^+$ which equals π^+ . Here, uniqueness and equivalence are in the sense that any policy $\hat{\pi}^+ \in \Pi_\mu$ that solves (18) induces the same completion distribution as π^+ for every prompt $x \in D$, i.e., $\mathbb{P}_{\hat{\pi}^+}(\cdot|x) = \mathbb{P}_{\pi^+}(\cdot|x)$.*

We now derive objective (18) and outline the proof of Theorem 1.

Derivation and Proof of Sufficiency. We first explain how we derive the critic-free objective (18) from the original advantage-based PMD (16) and show that the PMD solution π^+ minimizes the reformulated objective (18).

- Step 1: We rewrite (17) as the PMD optimality condition and eliminate the state-dependent partition function $Z_\mu(s_t)$. For any prompt x , response y , and step t with $1 \leq t \leq |y|$, we can rewrite (17) as

$$\log \pi^+(y_t|s_t) - \log \mu(y_t|s_t) - \eta A^\mu(s_t, y_t) + \log Z_\mu(s_t) = 0. \quad (20)$$

Taking the expectation over $y_t \sim \mu(\cdot|s_t)$ and using (10) gives

$$\log Z_\mu(s_t) = D_{\text{KL}}(\mu(\cdot|s_t) \parallel \pi^+(\cdot|s_t)) + \eta \sum_{y_t \in \mathcal{V}} \mu(y_t|s_t) A^\mu(s_t, y_t) = D_{\text{KL}}(\mu(\cdot|s_t) \parallel \pi^+(\cdot|s_t)). \quad (21)$$

Then, substituting (21) into (20) yields

$$\log \pi^+(y_t|s_t) - \log \mu(y_t|s_t) - \eta A^\mu(s_t, y_t) + D_{\text{KL}}(\mu(\cdot|s_t) \parallel \pi^+(\cdot|s_t)) = 0. \quad (22)$$

- Step 2: We next express the cumulative token-level advantages in terms of the terminal reward. By the Bellman equation, substituting (8) into (9) gives

$$A^\mu(s_t, y_t) = V^\mu(s_{t+1}) - V^\mu(s_t).$$

Summing over the trajectory gives

$$\sum_{t=1}^{|y|} A^\mu(s_t, y_t) = V^\mu(s_{|y|+1}) - V^\mu(s_1) = R(x, y) - V^\mu(x). \quad (23)$$

- Step 3: For any prompt-response pair (x, y) , summing (22) over the trajectory and using (23) gives

$$\sum_{t=1}^{|y|} (\log \pi^+(y_t|s_t) - \log \mu(y_t|s_t) + D_{\text{KL}}(\mu(\cdot|s_t) \parallel \pi^+(\cdot|s_t))) = \eta (R(x, y) - V^\mu(x)),$$

i.e.,

$$\delta(x, y; \pi^+, \mu) = 0. \quad (24)$$

Minimizing the expected squared residual under the rollout distribution gives (18).

Proof Sketch of Necessity. It remains to show that every optimal solution of (18) coincides with π^+ on all states reachable under μ . Define the token-level residual

$$d_\pi(s_t, y_t) = \eta A^\mu(s_t, y_t) - \left(\log \pi(y_t|s_t) - \log \mu(y_t|s_t) + D_{\text{KL}}(\mu(\cdot|s_t) \parallel \pi(\cdot|s_t)) \right).$$

For any feasible policy π for which the above quantities are finite, its conditional expectation under the rollout policy is zero:

$$\mathbb{E}_{y_t \sim \mu(\cdot|s_t)} [d_\pi(s_t, y_t)] = \eta \sum_{y_t \in \mathcal{V}} \mu(y_t|s_t) A^\mu(s_t, y_t) - \sum_{y_t \in \mathcal{V}} \mu(y_t|s_t) \log \frac{\pi(y_t|s_t)}{\mu(y_t|s_t)} - D_{\text{KL}}(\mu(\cdot|s_t) \parallel \pi(\cdot|s_t)) = 0.$$

Therefore, the cumulative token-level residuals form a martingale under the rollout policy μ . For any optimal solution of (18), the terminal value of this martingale equals the trajectory-level residual and is zero almost surely. Since the response length is bounded by $T_{\max}$, the martingale property then implies that every partial sum is zero almost surely. Thus, each token-level residual vanishes. The complete proof is provided in Appendix A.1.

3.2 Practical Approximation

We obtain the practical BPO loss (14) by approximating the critic-free objective (18). We linearize the squared-residual objective and estimate the initial value and prompt-dependent scaling factor from grouped rollouts. We then approximate the full reverse KL divergence using binary KL divergence and apply smoothing, masking, and clipping.

For response y^i , define the response-level loss $\mathcal{L}_i(\pi) = \phi(x) \delta(x, y^i; \pi, \mu)^2 / (2\eta)$. Its gradient can be decomposed into token-level contributions $\nabla \mathcal{L}_i(\pi) = \sum_{t=1}^{|y^i|} \nabla \mathcal{L}_{i,t}(\pi)$. Throughout this section, ∇ denotes the gradient with respect to the parameters of π with μ fixed.

- Step 1: linearized approximation. We linearize the squared-residual loss with respect to the residual δ , around its value at $\pi = \mu$. Since $\delta(x, y^i; \mu, \mu) = \eta (R(x, y^i) - V^\mu(x))$, this replaces the residual factor $\delta(x, y^i; \pi, \mu)/\eta$ in the gradient by $R(x, y^i) - V^\mu(x)$. For response y^i and the corresponding state $s_t^i = (x, y_{<t}^i)$, the per-token gradient contribution is approximated by

$$\begin{aligned} \nabla \mathcal{L}_{i,t}(\pi) &= -\phi(x) \frac{\delta(x, y^i; \pi, \mu)}{\eta} \nabla \left(\log \pi(y_t^i|s_t^i) + D_{\text{KL}}(\mu(\cdot|s_t^i) \parallel \pi(\cdot|s_t^i)) \right) \\ &\approx -\phi(x) \left(R(x, y^i) - V^\mu(x) \right) \nabla \left(\log \pi(y_t^i|s_t^i) + D_{\text{KL}}(\mu(\cdot|s_t^i) \parallel \pi(\cdot|s_t^i)) \right). \end{aligned} \quad (25)$$

- Step 2: group-based estimation and normalization. The initial value function $V^\mu(x) = \mathbb{E}_{y \sim \mathbb{P}_\mu(\cdot|x)} [R(x, y)]$ has the unbiased estimator $\text{mean}(\{R(x, y^j)\}_{j=1}^G)$. We further choose $\phi(x)$ as the inverse reward standard deviation, i.e., $\phi(x) = \frac{1}{\sqrt{\text{Var}_{y \sim \mathbb{P}_\mu(\cdot|x)}(R(x, y))}}$. We replace it by its empirical counterpart $\phi(x) \approx \frac{1}{\text{std}(\{R(x, y^i)\}_{i=1}^G)}$. With these substitutions, (25) becomes

$$\nabla \mathcal{L}_{i,t}(\pi) \approx -\hat{A}^i \nabla \left(\log \pi(y_t^i|s_t^i) + D_{\text{KL}}(\mu(\cdot|s_t^i) \parallel \pi(\cdot|s_t^i)) \right). \quad (26)$$

- Step 3: binary KL approximation and additive smoothing. Computing the full reverse KL divergence in (26) can be costly. We therefore replace the full KL divergence $D_{\text{KL}}(\mu(\cdot|s_t^i) \parallel \pi(\cdot|s_t^i))$ in (26) with the binary KL divergence $D_{\text{KL}}^{\text{bin}}(\mu(\cdot|s_t^i) \parallel \pi(\cdot|s_t^i); y_t^i)$ as defined in Section 2.6. As proved in Appendix A.2, the following identity holds

$$\nabla \left(\log \pi(y_t^i|s_t^i) + D_{\text{KL}}^{\text{bin}}(\mu(\cdot|s_t^i) \parallel \pi(\cdot|s_t^i); y_t^i) \right) = \frac{1 - \mu(y_t^i|s_t^i)}{1 - \pi(y_t^i|s_t^i)} \nabla \log \pi(y_t^i|s_t^i). \quad (27)$$

Substituting (27) into (26) gives

$$\nabla \mathcal{L}_{i,t}(\pi) \approx -\hat{A}^i \frac{1 - \mu(y_t^i | s_t^i)}{1 - \pi(y_t^i | s_t^i)} \nabla \log \pi(y_t^i | s_t^i). \quad (28)$$

The multiplier $\frac{1 - \mu(y_t^i | s_t^i)}{1 - \pi(y_t^i | s_t^i)}$ in (28) can become large when $\pi(y_t^i | s_t^i)$ approaches one. We use the additively smoothed mismatch-correction weight ω_t^i defined in (15) for numerical stability. The resulting per-token gradient is

$$\nabla \mathcal{L}_{i,t}(\pi) \approx -\hat{A}^i \omega_t^i \nabla \log \pi(y_t^i | s_t^i). \quad (29)$$

• Step 4: masking and clipping. The preceding steps yield the approximate per-token gradient direction $-\hat{A}^i \omega_t^i \nabla \log \pi(y_t^i | s_t^i)$.

We apply the GRPO-style clipping mask M_t^i and cap ω_t^i at C . The resulting per-token BPO loss gradient is

$$\nabla \mathcal{L}_{i,t}^{\text{BPO}}(\pi) = -\hat{A}^i M_t^i \min\{\omega_t^i, C\} \nabla \log \pi(y_t^i | s_t^i).$$

This gives the BPO loss (14).

4 Experiments

4.1 Experimental Setup

We evaluate BPO on mathematical reasoning with Qwen3-30B-A3B-Base [3]. All methods are trained on the English subset of DAPO-Math-17k [4]. We compare BPO with GRPO-ClipHigher [1, 4], GSPO [7], CISPO [13], and DPPO [8]. We conduct controlled comparisons by varying only the policy loss and keeping all other experimental settings identical.

Each rollout batch contains 256 prompts with 16 responses per prompt. The resulting 4096 responses are split into 8 minibatches of 512 responses. One training step consists of one rollout batch followed by its eight minibatch updates. We train each method for 400 training steps, corresponding to 3200 optimizer updates. The maximum response length is 16384 tokens. All methods use rollout-router replay (R3) [34]. BPO uses $\epsilon = 0.1$ and $C = 3.0$. Ablations on these hyperparameters are presented in Appendix C. More details are given in Appendix B.

We evaluate on AIME24, AIME25, and AIME26 [26]. We estimate Pass@1 using Avg@32: we sample 32 responses per question, average their correctness, and then average over questions. The average accuracy is the arithmetic mean of the three benchmark scores. Table 1 reports each method’s results at the checkpoint with the highest mean accuracy across the three benchmarks.

4.2 Main Results

Table 1 shows that BPO achieves 50.5% average accuracy, compared with 39.5% for GRPO-ClipHigher and 47.4% for CISPO, the strongest baseline. These correspond to gains of 11.0 and 3.1 percentage points, respectively. BPO also achieves the highest accuracy on all three benchmarks.

BPO achieves the highest final accuracy (Figure 1). After 400 training steps, its average accuracy is 49.4%, compared with 45.5% for DPPO, the strongest baseline at the end of training.

Table 1: AIME Avg@32 (%) on Qwen3-30B-A3B-Base. Each row reports scores at the checkpoint with the highest mean accuracy across the three benchmarks. The best result in each column is bold.

Method	AIME24	AIME25	AIME26	Avg.
GRPO-ClipHigher	45.6	34.8	38.0	39.5
GSPO	50.3	35.5	44.6	43.5
CISPO	52.7	39.0	50.4	47.4
DPPO	55.8	39.2	44.2	46.4
BPO	57.4	41.0	53.0	50.5

5 Conclusion

We introduced Bellman Policy Optimization (BPO), a critic-free method for RLVR derived from Policy Mirror Descent. Using the Bellman equations, we obtained a trajectory-level objective that avoids estimating intermediate state values. We proved that this objective and the original PMD objective have the same unique optimal solution on states reachable under the rollout policy. We then derived a practical token-level loss through approximations, with a smoothed mismatch-correction weight replacing the importance-sampling ratio. On Qwen3-30B-A3B-Base, BPO achieves a peak average accuracy of 50.5% across AIME 2024–2026, outperforming GRPO-ClipHigher, GSPO, CISPO, and DPPO by 3.1–11.0 percentage points. Ablations on Qwen3-4B-Base show similar performance across a range of smoothing and truncation settings.

Acknowledgements

We thank Sam, Lingfeng, Chris, Simon, Beibin, Yifan, Charlotte, Ziven, Robert, Yuzhen, Yingying, Xingxuan, Zhenwen, Feng, Kaiyu, Kevin, Chen, Chencheng, Ziran, and Marcus for helpful discussions. ChatGPT and Claude were used to edit and proofread the language of this manuscript. Codex and Claude Code were used to assist with debugging code.

References

- [1] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [2] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [3] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [4] Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *Advances in Neural Information Processing Systems*, 38:113222–113244, 2026.

- [5] Marah Abdin, Sahaj Agarwal, Ahmed Awadallah, Vidhisha Balachandran, Harkirat Behl, Lingjiao Chen, Gustavo de Rosa, Suriya Gunasekar, Mojan Javaheripi, Neel Joshi, et al. Phi-4-reasoning technical report. *arXiv preprint arXiv:2504.21318*, 2025.
- [6] Xumeng Wen, Zihan Liu, Shun Zheng, Shengyu Ye, Zhirong Wu, Yang Wang, Zhijian Xu, Xiao Liang, Junjie Li, Ziming Miao, et al. Reinforcement learning with verifiable rewards implicitly incentivizes correct reasoning in base llms. In *International Conference on Learning Representations*, volume 2026, pages 49450–49483, 2026.
- [7] Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.
- [8] Penghui Qi, Xiangxin Zhou, Zichen Liu, Tianyu Pang, Chao Du, Min Lin, and Wee Sun Lee. Rethinking the trust region in llm reinforcement learning. *arXiv preprint arXiv:2602.04879*, 2026.
- [9] Chiyu Ma, Shuo Yang, Kexin Huang, Jinda Lu, Haoming Meng, Shangshang Wang, Bolin Ding, Soroush Vosoughi, Guoyin Wang, and Jingren Zhou. Fipo: Eliciting deep reasoning with future-kl influenced policy optimization. *arXiv preprint arXiv:2603.19835*, 2026.
- [10] Renjie Mao, Xiangxin Zhou, Lvfang Tao, Yixin Ding, Yu Shi, Yongguang Lin, Yuheng Wu, Honglin Zhu, Qian Qiu, and Wenxi Zhu. Beyond uniform token-level trust region in llm reinforcement learning. *arXiv preprint arXiv:2606.10968*, 2026.
- [11] Zhewei Kang, Aosong Feng, Sergey Levine, Dawn Song, and Xuandong Zhao. Vimpo: Value-implicit policy optimization for llms. *arXiv preprint arXiv:2606.20008*, 2026.
- [12] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding rl-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- [13] Aili Chen, Aonian Li, Bangwei Gong, Binyang Jiang, Bo Fei, Bo Yang, Boji Shan, Changqing Yu, Chao Wang, Cheng Zhu, et al. Minimax-m1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025.
- [14] Yifan Zhang, Yifeng Liu, Rina Hughes, Yang Yuan, Quanquan Gu, and Andrew Yao. On the design of kl-regularized policy gradient algorithms for llm reasoning. In *International Conference on Learning Representations*, volume 2026, pages 88357–88395, 2026.
- [15] Mingjie Liu, Shizhe Diao, Ximing Lu, Jian Hu, Xin Dong, Yejin Choi, Jan Kautz, and Yi Dong. Prorl: Prolonged reinforcement learning expands reasoning boundaries in large language models. *Advances in Neural Information Processing Systems*, 38:17998–18031, 2026.
- [16] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [17] Aaron Blakeman, Aaron Grattafiori, Aarti Basant, Abhibha Gupta, Abhinav Khattar, Adi Renduchintala, Aditya Vavre, Akanksha Shukla, Akhiad Bercovich, Aleksander Ficek, et al. Nvidia nemotron 3: Efficient and open intelligence. *arXiv preprint arXiv:2512.20856*, 2025.

- [18] Aaron Blakeman, Aaron Thomas, Aastha Jhunjhunwala, Abhibha Gupta, Abhinav Khattar, Adam Rajfer, Adi Renduchintala, Adil Asif, Aditya Vavre, Adriana Flores Miranda, et al. Nemotron 3 ultra: Open, efficient mixture-of-experts hybrid mamba-transformer model for agentic reasoning. *arXiv preprint arXiv:2606.15007*, 2026.
- [19] Ling Team, Anqi Shen, Baihui Li, Bin Hu, Bin Jing, Cai Chen, Chao Huang, Chao Zhang, Chaokun Yang, Cheng Lin, et al. Every step evolves: Scaling reinforcement learning for trillion-scale thinking model. *arXiv preprint arXiv:2510.18855*, 2025.
- [20] Guanghui Lan. Policy mirror descent for reinforcement learning: Linear convergence, new sampling complexity, and generalized problem classes. *Mathematical programming*, 198(1):1059–1106, 2023.
- [21] Lin Xiao. On the convergence rates of policy gradient methods. *Journal of Machine Learning Research*, 23(282):1–36, 2022.
- [22] Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron Courville, and Nicolas Le Roux. VinePPO: Refining credit assignment in RL training of LLMs. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 29557–29590. PMLR, 13–19 Jul 2025.
- [23] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. Direct preference optimization: your language model is secretly a reward model. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, pages 53728–53741, 2023.
- [24] Yongcheng Zeng, Guoqing Liu, Weiyu Ma, Ning Yang, Haifeng Zhang, and Jun Wang. Token-level direct preference optimization. In *International Conference on Machine Learning*, pages 58348–58365. PMLR, 2024.
- [25] Rafael Rafailov, Joey Hejna, Ryan Park, and Chelsea Finn. From r to q^* : Your language model is secretly a q -function. In *First Conference on Language Modeling*.
- [26] Art of Problem Solving. AIME problems and solutions.
- [27] Chujie Zheng, Kai Dang, Bowen Yu, Mingze Li, Huiqiang Jiang, Junrong Lin, Yuqiong Liu, Hao Lin, Chencan Wu, Feng Hu, et al. Stabilizing reinforcement learning with llms: Formulation and practices. *arXiv preprint arXiv:2512.01374*, 2025.
- [28] Penghui Qi, Zichen Liu, Xiangxin Zhou, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Defeating the training-inference mismatch via fp16. *arXiv preprint arXiv:2510.26788*, 2025.
- [29] Jiarui Yao, Xiangxin Zhou, Penghui Qi, Wee Sun Lee, Liefeng Bo, and Tianyu Pang. Rethinking the divergence regularization in llm rl. *arXiv preprint arXiv:2606.09821*, 2026.
- [30] Feng Yao, Liyuan Liu, Dinghuai Zhang, Chengyu Dong, Jingbo Shang, and Jianfeng Gao. On the rollout-training mismatch in modern rl systems. In *OPT 2025: Optimization for Machine Learning*, 2025.
- [31] Richard S Sutton, Andrew G Barto, and Andrew Barto. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.

- [32] Matthieu Geist, Bruno Scherrer, and Olivier Pietquin. A theory of regularized markov decision processes. In *International conference on machine learning*, pages 2160–2169. PMLR, 2019.
- [33] Manan Tomar, Lior Shani, Yonathan Efroni, and Mohammad Ghavamzadeh. Mirror descent policy optimization. *arXiv preprint arXiv:2005.09814*, 2020.
- [34] Wenhan Ma, Hailin Zhang, Liang Zhao, Yifan Song, Yudong Wang, Zhifang Sui, and Fuli Luo. Stabilizing moe reinforcement learning by aligning training and inference routers. *arXiv preprint arXiv:2510.11370*, 2025.
- [35] Hamish Ivison, Junjie Oscar Yin, Rulin Shao, Teng Xiao, Nathan Lambert, and Hannaneh Hajishirzi. Tmax: A simple recipe for terminal agents. *arXiv preprint arXiv:2606.23321*, 2026.
- [36] Devvrit Khatri, Lovish Madaan, Rishabh Tiwari, Rachit Bansal, Venkata Sai Surya Subramanyam Duvvuri, Manzil Zaheer, Inderjit Dhillon, David Brandfonbrener, and Rishabh Agarwal. The art of scaling reinforcement learning compute for llms. In *International Conference on Learning Representations*, volume 2026, pages 72438–72467, 2026.

A Omitted Proofs

A.1 Proof of Theorem 1

Sufficiency follows from (24). It remains to prove necessity. We use $\mathbb{I}_A$ to denote the indicator of an event A throughout this section.

Let $\text{supp}\mu(\cdot|s)$ denote the set of actions with positive probability under policy μ at state s . Let $\text{supp}\mathbb{P}_\mu(\cdot|x)$ denote the set of responses with positive probability under policy μ given prompt x . Let $\mathcal{S}_x^\mu$ denote the set of states reachable from prompt x with positive probability under policy μ .

Fix any prompt $x \in D$. For any state-action pair (s, a) , define

$$d_\pi(s, a) = \eta A^\mu(s, a) - \left(\log \pi(a|s) - \log \mu(a|s) + D_{\text{KL}}(\mu(\cdot|s) \parallel \pi(\cdot|s)) \right).$$

By definition,

$$\delta(x, y; \pi, \mu) = \sum_{t=1}^{|y|} d_\pi(s_t, y_t). \quad (30)$$

We first show that the minimum of (18) is zero. The PMD optimality condition (22) gives

$$d_{\pi^+}(s, a) = 0.$$

The exponential-form update (17) preserves the support of the rollout policy, so $\pi^+ \in \Pi_\mu$ and

$$\delta(x, y; \pi^+, \mu) = 0, \quad \forall y \in \text{supp}\mathbb{P}_\mu(\cdot|x).$$

Hence, π^+ is feasible for (18) and achieves objective value zero. Since the objective is nonnegative, the minimum value of (18) is zero.

Now let $\hat{\pi}^+ \in \Pi_\mu$ be any optimal solution of (18). Since the minimum value is zero and $\phi(x) > 0$, we have

$$\delta(x, Y; \hat{\pi}^+, \mu) = 0, \quad Y \sim \mathbb{P}_\mu(\cdot|x)\text{-almost surely.} \quad (31)$$

Since the objective value is finite, for every state $s \in \mathcal{S}_x^\mu$, $D_{\text{KL}}(\mu(\cdot|s) \parallel \hat{\pi}^+(\cdot|s)) < \infty$. Consequently, $\mu(\cdot|s)$ is absolutely continuous with respect to $\hat{\pi}^+(\cdot|s)$ for any $s \in \mathcal{S}_x^\mu$. Thus, all quantities below are finite on states reachable under μ .

Let τ be the time at which generation terminates:

$$\tau = \inf \{t \geq 1 : Y_t = \text{EOS}\} \wedge T_{\max}.$$

Define the filtration

$$\mathcal{F}_t = \sigma(Y_1, \dots, Y_{t \wedge \tau}), \quad 0 \leq t \leq T_{\max}.$$

For $1 \leq t \leq T_{\max}$, define

$$D_t = \mathbb{I}_{\{t \leq \tau\}} d_{\hat{\pi}^+}(S_t, Y_t), \quad M_n = \sum_{t=1}^n D_t,$$

where $S_t = (x, Y_{<t})$ on the event $\{t \leq \tau\}$.

Since $\mathbb{I}_{\{t \leq \tau\}}$ and S_t are $\mathcal{F}_{t-1}$ -measurable, we have

$$\begin{aligned} & \mathbb{E}[D_t \mid \mathcal{F}_{t-1}] \\ &= \mathbb{I}_{\{t \leq \tau\}} \sum_{a \in \mathcal{V}} \mu(a|S_t) d_{\hat{\pi}^+}(S_t, a) \\ &= \mathbb{I}_{\{t \leq \tau\}} \eta \sum_{a \in \mathcal{V}} \mu(a|S_t) A^\mu(S_t, a) \\ &\quad - \mathbb{I}_{\{t \leq \tau\}} \left(\sum_{a \in \mathcal{V}} \mu(a|S_t) \log \frac{\hat{\pi}^+(a|S_t)}{\mu(a|S_t)} + D_{\text{KL}}(\mu(\cdot|S_t) \parallel \hat{\pi}^+(\cdot|S_t)) \right) \\ &= 0, \end{aligned}$$

where the last equality follows from (10). Thus, $\{M_n\}_{n=0}^{T_{\max}}$ is an integrable martingale.

By (30) and (31),

$$M_{T_{\max}} = \sum_{t=1}^{\tau} d_{\hat{\pi}^+}(S_t, Y_t) = \delta(x, Y; \hat{\pi}^+, \mu) = 0, \quad \mathbb{P}_\mu(\cdot|x)\text{-almost surely.}$$

Thus, for any $0 \leq n \leq T_{\max}$,

$$M_n = \mathbb{E}[M_{T_{\max}} | \mathcal{F}_n] = 0, \quad \mathbb{P}_\mu(\cdot|x)\text{-almost surely.}$$

It follows that for any $1 \leq t \leq T_{\max}$,

$$D_t = M_t - M_{t-1} = 0 \quad \mathbb{P}_\mu(\cdot|x)\text{-almost surely.}$$

Consider any state $s \in \mathcal{S}_x^\mu$ and any action $a \in \text{supp} \mu(\cdot|s)$. For the corresponding step t , the event $\{S_t = s, Y_t = a\}$ has strictly positive probability under $\mathbb{P}_\mu(\cdot|x)$. Since $D_t = 0$ almost surely, this implies

$$d_{\hat{\pi}^+}(s, a) = 0.$$

Combining this with $d_{\pi^+}(s, a) = 0$, we obtain

$$\log \frac{\hat{\pi}^+(a|s)}{\pi^+(a|s)} = D_{\text{KL}}(\mu(\cdot|s) \parallel \pi^+(\cdot|s)) - D_{\text{KL}}(\mu(\cdot|s) \parallel \hat{\pi}^+(\cdot|s)).$$

The right-hand side does not depend on a . Hence, for every $s \in \mathcal{S}_x^\mu$, there exists a constant $c(s) > 0$ such that

$$\hat{\pi}^+(a|s) = c(s)\pi^+(a|s), \quad \forall a \in \text{supp}\mu(\cdot|s).$$

Since $\hat{\pi}^+, \pi^+ \in \Pi_\mu$, both policies assign zero probability outside $\text{supp}\mu(\cdot|s)$. Therefore,

$$1 = \sum_{a \in \text{supp}\mu(\cdot|s)} \hat{\pi}^+(a|s) = c(s) \sum_{a \in \text{supp}\mu(\cdot|s)} \pi^+(a|s) = c(s).$$

Thus, $c(s) = 1$, and

$$\hat{\pi}^+(\cdot|s) = \pi^+(\cdot|s), \quad \forall s \in \mathcal{S}_x^\mu.$$

In addition, since $\hat{\pi}^+, \pi^+ \in \Pi_\mu$, both π^+ and $\hat{\pi}^+$ assign zero probability outside $\text{supp}\mathbb{P}_\mu(\cdot|x)$. Consequently,

$$\mathbb{P}_{\hat{\pi}^+}(\cdot|x) = \mathbb{P}_{\pi^+}(\cdot|x).$$

This completes the proof. □

A.2 Proof of the Identity (27)

Fix a state s and an action y , and let

$$p = \pi(y|s), \quad q = \mu(y|s).$$

The binary KL divergence associated with action y is

$$D_{\text{KL}}^{\text{bin}}(\mu(\cdot|s) \parallel \pi(\cdot|s); y) = q \log \frac{q}{p} + (1 - q) \log \frac{1 - q}{1 - p}.$$

Differentiating with respect to π gives

$$\nabla D_{\text{KL}}^{\text{bin}}(\mu(\cdot|s) \parallel \pi(\cdot|s); y) = -q \nabla \log p + \frac{1 - q}{1 - p} \nabla p.$$

Using the fact that $\nabla p = p \nabla \log p$, we obtain

$$\nabla \left(\log p + D_{\text{KL}}^{\text{bin}}(\mu(\cdot|s) \parallel \pi(\cdot|s); y) \right) = (1 - q) \nabla \log p + \frac{(1 - q)p}{1 - p} \nabla \log p = \frac{1 - \mu(y|s)}{1 - \pi(y|s)} \nabla \log \pi(y|s),$$

i.e.,

$$\nabla \left(\log \pi(y|s) + D_{\text{KL}}^{\text{bin}}(\mu(\cdot|s) \parallel \pi(\cdot|s); y) \right) = \frac{1 - \mu(y|s)}{1 - \pi(y|s)} \nabla \log \pi(y|s).$$

This proves (27). □

B Experimental Details

All experiments use AdamW with a constant learning rate of 10^{-6} , $\beta_1 = 0.9$, $\beta_2 = 0.98$, and weight decay 0.1. The gradient clipping threshold is 1.0. We use no KL penalty and the entropy bonus coefficient is zero. Training rollouts are sampled at temperature 1.0 and top- $p = 1.0$, with top- k filtering disabled. Advantages are centered and normalized by the reward standard deviation within each response group as in GRPO. We use `seq-mean-token-mean` for loss aggregation. These settings are shared across methods.

We use 400 training steps for the main experiments in Section 4 and 1000 for the ablations in Appendix C. The AIME benchmarks are available on Hugging Face.¹ We evaluate every ten rollout batches (i.e., training steps). All evaluations use 32 responses per question at temperature 1.0 and top- $p = 1.0$, with top- k filtering disabled. Evaluation uses the same response-length limit as training for each model.

We report Avg@32 as an estimate of Pass@1. The Avg. column and the plotted average accuracy are the arithmetic mean of the AIME24, AIME25, and AIME26 Avg@32 scores.

We select the checkpoint with the highest mean Avg@32 across the three benchmarks, choosing the earlier step in a tie.

GRPO-ClipHigher uses the asymmetric clipping interval $[0.8, 1.28]$ from DAPO [4]. CISPO uses an importance-weight cap of 3.0, following the configuration used in the stability experiments of DPPO [8]. DPPO uses binary total variation with threshold $\delta = 0.1$, following TMax [35]. GSPO uses the clipping interval $[1 - 3 \times 10^{-3}, 1 + 5 \times 10^{-3}]$, following the ablation results in [36]. BPO uses the default setting $\epsilon = 0.1$ and $C = 3.0$ in the main experiments.

C Ablation Studies

We study the two BPO hyperparameters ϵ and C on Qwen3-4B-Base with 1000 training steps. Here, each batch contains 128 prompts with 16 responses per prompt. The 2048 responses are split into four minibatches of 512, with a maximum response length of 8192 tokens. One training step in the figures denotes one rollout batch followed by its minibatch updates. Each sweep changes only one hyperparameter while keeping the other settings fixed. Both sweeps in Section C.1 and Section C.2 use the training setup in Appendix B and share the same GRPO-ClipHigher baseline.

C.1 Sensitivity to ϵ

We fix $C = 3.0$ and vary ϵ over $\{0.05, 0.1, 0.2, 0.3\}$. Figure 2 shows the training curves, and Table 2 summarizes the results.

BPO achieves similar average accuracies of 25.4%–25.8% for $\epsilon \in \{0.05, 0.1, 0.2\}$. At $\epsilon = 0.3$, accuracy is 24.1%, still 3.6 percentage points above GRPO-ClipHigher. BPO shows little sensitivity to ϵ from 0.05 to 0.2 and outperforms the baseline at all four settings.

¹AIME24: <https://huggingface.co/datasets/math-ai/aime24>, AIME25: <https://huggingface.co/datasets/math-ai/aime25>, AIME26: <https://huggingface.co/datasets/math-ai/aime26>.

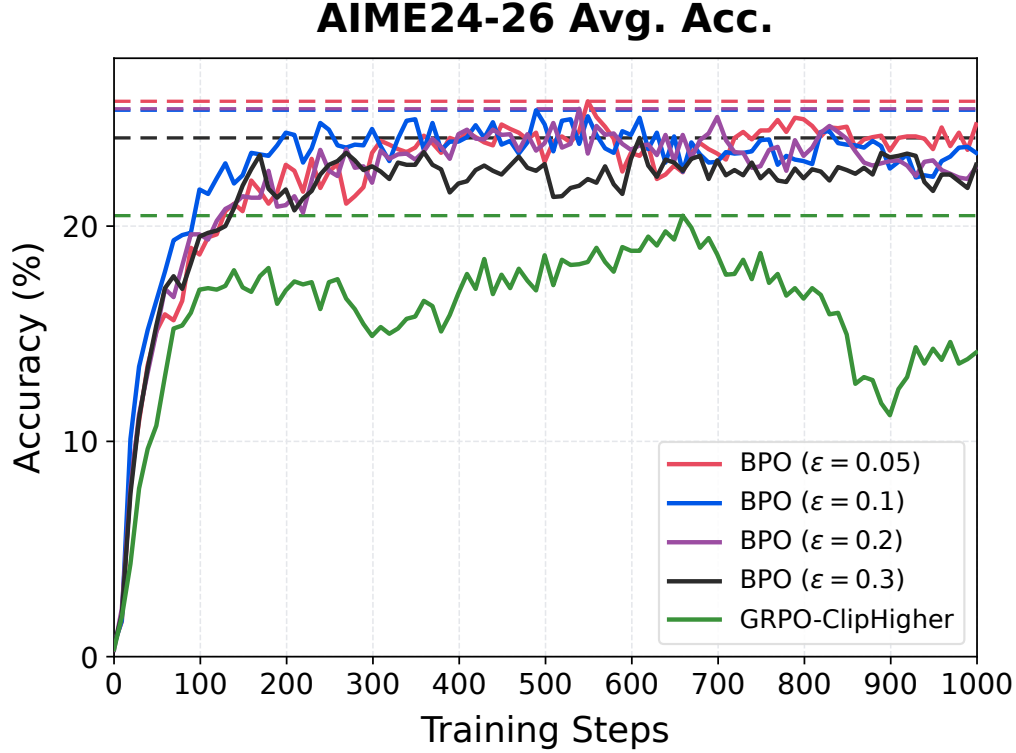


Figure 2: Effect of BPO’s smoothing parameter ϵ on Qwen3-4B-Base with $C = 3.0$. Curves show mean Avg@32 across AIME24–26. Dashed lines mark the peak of each curve.

Table 2: Ablation on BPO’s smoothing parameter ϵ with $C = 3.0$, reporting AIME Avg@32 (%). Each row reports scores at the checkpoint with the highest mean accuracy across the three benchmarks. The best result in each column is bold.

Method	AIME24	AIME25	AIME26	Avg.
BPO ($\epsilon = 0.05$)	31.6	23.0	22.8	25.8
BPO ($\epsilon = 0.1$)	27.6	26.8	21.8	25.4
BPO ($\epsilon = 0.2$)	29.1	24.3	23.0	25.5
BPO ($\epsilon = 0.3$)	26.4	25.5	20.4	24.1
GRPO-ClipHigher	22.1	22.8	16.6	20.5

C.2 Sensitivity to C

We fix $\epsilon = 0.1$ and vary C over $\{2.0, 3.0, 4.0\}$. Figure 3 shows the training curves, and Table 3 summarizes the results.

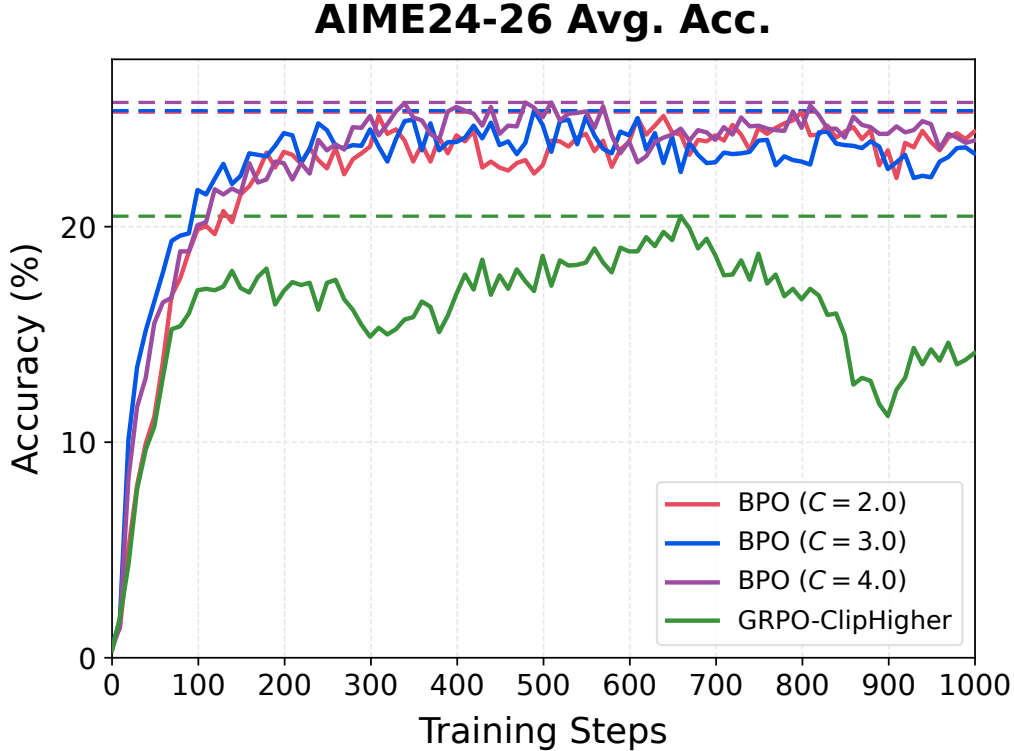


Figure 3: Effect of BPO’s truncation parameter C on Qwen3-4B-Base with $\epsilon = 0.1$. Curves show mean Avg@32 across AIME24–26. Dashed lines mark the peak of each curve.

Table 3: Ablation on BPO’s truncation parameter C with $\epsilon = 0.1$, reporting AIME Avg@32 (%). Each row reports scores at the checkpoint with the highest mean accuracy across the three benchmarks. The best result in each column is bold.

Method	AIME24	AIME25	AIME26	Avg.
BPO ($C = 2.0$)	26.4	26.0	23.5	25.3
BPO ($C = 3.0$)	27.6	26.8	21.8	25.4
BPO ($C = 4.0$)	27.1	29.0	21.3	25.8
GRPO-ClipHigher	22.1	22.8	16.6	20.5

The three values of C yield average accuracies between 25.3% and 25.8%, a spread of 0.5 percentage points. All three exceed the GRPO-ClipHigher baseline of 20.5%. These results show that BPO is insensitive to C from 2.0 to 4.0.